Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»
Филиал «Минский радиотехнический колледж»

Учебный предмет
«Инструментальное программное обеспечение»

Отчёт
по выполнению лабораторной работы №15-16
«Разработка, отладка и испытание программ с применением свойств классов»

Выполнила: учащаяся гр. 4к9392
Кривнюк Д.А.
Проверила: Лазовская Н.В.

Минск,2025

Лабораторная работа №15-16

Тема работы: «Разработка, отладка и испытание программ с применением свойств классов»

1. Цель работы

Научить применять свойства при проектировании классов и разработке программ.

2. Задание

В классы из лабораторной работы № 14 добавить не менее 3 свойств и двух декораторов.

3. Оснащение работы

Задание по варианту, ЭВМ, среда разработки **Python 3.7, IDLE**.

4. Порядок выполнения работы

Код программы:

```
from abc import ABC, abstractmethod import datetime

class Car(ABC):
    def __init__(self, brand, color, mileage=0, year=2020, fuel_level=50,
owner="Неизвестен"):
        self.brand = brand         self.color = color         self.mileage = mileage
self.year = year         self.fuel_level = fuel_level         self.owner = owner

    def info(self):
        return (f"Марка: {self.brand}, Цвет: {self.color}, Пробег: {self.mileage} км, "
f"Год выпуска: {self.year}, Топливо: {self.fuel_level} л, Владелец: {self.owner}")

    def __str__(self):
        return self.info()

    def __add__(self, distance):
        if isinstance(distance, (int, float)):
            self.mileage += distance             return self         return NotImplemented

    def drive(self, distance):
        if self.fuel_level <= 0:
            return "Нет топлива для поездки!"         self.mileage += distance
self.fuel_level -= distance * 0.1         return f"Машина проехала {distance} км. Общий про-
бег: {self.mileage} км. Остаток топлива: {self.fuel_level:.1f} л"

    def is_old(self):
        if self.mileage > 100000:
            return "У машины большой пробег."         else:
            return "У машины небольшой пробег."

    @staticmethod     def convert_km_to_miles(km):
        return km * 0.621371

    @staticmethod     def get_vehicle_type():
        return "Автомобиль"

    @property     def age(self):
        current_year = datetime.datetime.now().year         return current_year - self.year
```

```
    @classmethod     def from_dict(cls, data: dict):
        return cls(
            brand=data.get("brand", "Unknown"),
            color=data.get("color", "Black"),
            mileage=data.get("mileage", 0),
            year=data.get("year", 2020),
            fuel_level=data.get("fuel_level", 50),
            owner=data.get("owner", "Неизвестен")
        )

    @abstractmethod     def get_engine_type(self):
        pass

    @abstractmethod     def calculate_tax(self):
        pass


class PassengerCar(Car):
    def __init__(self, brand, color, mileage=0, engine_type="Бензиновый", year=2020,
fuel_level=50, owner="Неизвестен"):
        super().__init__(brand, color, mileage, year, fuel_level, owner)
        self.engine_type = engine_type

    def get_engine_type(self):
        return f"Тип двигателя: {self.engine_type}"

    def calculate_tax(self):
        base_tax = 5000         if self.mileage > 150000:
            return base_tax * 0.8         return base_tax

    def info(self):
        return super().info() + f", Тип двигателя: {self.engine_type}"


class Truck(Car):
    def __init__(self, brand, color, mileage=0, engine_type="Дизельный", load_capacity=10,
                 year=2020, fuel_level=50, owner="Неизвестен"):
        super().__init__(brand, color, mileage, year, fuel_level, owner)
        self.engine_type = engine_type         self.load_capacity = load_capacity

    def get_engine_type(self):
        return f"Тип двигателя: {self.engine_type}"

    def calculate_tax(self):
        return 15000 + self.load_capacity * 100

    def info(self):
        return super().info() + f", Грузоподъемность: {self.load_capacity}т"

    def load_cargo(self):
        return f"{self.brand}: Погружено {self.load_capacity} тонн груза"


class Motorcycle(Car):
    def __init__(self, brand, color, mileage=0, engine_type="Бензиновый",
                 year=2020, fuel_level=20, owner="Неизвестен"):
        super().__init__(brand, color, mileage, year, fuel_level, owner)
        self.engine_type = engine_type

    def get_engine_type(self):
        return f"Тип двигателя: {self.engine_type}"

    def calculate_tax(self):
```

```
        return 2000

    def info(self):
        return super().info() + f", Тип: Мотоцикл"

    def do_wheelie(self):
        return f"{self.brand}: Мотоцикл делает вилли!"


class Bus(Car):
    def __init__(self, brand, color, mileage=0, engine_type="Дизельный", passenger_capac-
ity=50,
                 year=2020, fuel_level=100, owner="Неизвестен"):
        super().__init__(brand, color, mileage, year, fuel_level, owner)
        self.engine_type = engine_type         self.passenger_capacity = passenger_capacity

    def get_engine_type(self):
        return f"Тип двигателя: {self.engine_type}"

    def calculate_tax(self):
        return 12000 + self.passenger_capacity * 50

    def info(self):
        return super().info() + f", Вместимость: {self.passenger_capacity} пассажиров"

    def announce_stop(self):
        return f"{self.brand}: Следующая остановка!"


print("Добро пожаловать в систему учета транспортных средств!")

while True:
    print("\nСОЗДАНИЕ ТРАНСПОРТНОГО СРЕДСТВА")

    print("Выберите тип транспортного средства:")
    print("1. Легковой автомобиль")
    print("2. Грузовик")
    print("3. Мотоцикл")
    print("4. Автобус")
    print("0. Выйти")

    type_choice = input("Ваш выбор: ")

    if type_choice == "0":
        print("Выход из программы.")
        break

    print("\nВведите данные о транспорте:")
    brand = input("Марка: ")
    color = input("Цвет: ")
    mileage = int(input("Пробег (в км): "))
    year = int(input("Год выпуска: "))
    fuel_level = float(input("Уровень топлива (л): "))
    owner = input("Владелец: ")

    if type_choice == "1":
        engine_type = input("Тип двигателя: ")
        vehicle = PassengerCar(brand, color, mileage, engine_type, year, fuel_level, owner)
    elif type_choice == "2":
        engine_type = input("Тип двигателя: ")
        load_capacity = float(input("Грузоподъемность (тонн): "))
        vehicle = Truck(brand, color, mileage, engine_type, load_capacity, year, fuel_level,
owner)
    elif type_choice == "3":
```

```
        engine_type = input("Тип двигателя: ")
        vehicle = Motorcycle(brand, color, mileage, engine_type, year, fuel_level, owner)
    elif type_choice == "4":
        engine_type = input("Тип двигателя: ")
        passenger_capacity = int(input("Вместимость пассажиров: "))
        vehicle = Bus(brand, color, mileage, engine_type, passenger_capacity, year,
fuel_level, owner)
    else:
        print("Неверный выбор")
        continue

    print(f"\nТранспортное средство создано: {vehicle.info()}")

    while True:
        print(f"\nМЕНЮ ДЕЙСТВИЙ: {vehicle.brand}")
        print("1. Проехать расстояние")
        print("2. Проверить пробег")
        print("3. Показать информацию")
        print("4. Конвертировать пробег в мили")
        print("5. Рассчитать налог")
        print("6. Показать тип двигателя")
        print("7. Дополнительное действие")
        print("8. Показать возраст машины")
        print("9. Показать уровень топлива")
        print("10. Создать новое транспортное средство")
        print("0. Выйти из программы")

        action = input("Ваш выбор: ")

        if action == "1":
            distance = int(input("Сколько км проехать? "))
            print(vehicle.drive(distance))
        elif action == "2":
            print(vehicle.is_old())
        elif action == "3":
            print(vehicle.info())
        elif action == "4":
            miles = Car.convert_km_to_miles(vehicle.mileage)
            print(f"Пробег {vehicle.mileage} км = {miles:.2f} миль")
        elif action == "5":
            tax = vehicle.calculate_tax()
            print(f"Налог на транспорт: {tax} руб.")
        elif action == "6":
            print(vehicle.get_engine_type())
        elif action == "7":
            if isinstance(vehicle, Truck):
                print(vehicle.load_cargo())
            elif isinstance(vehicle, Motorcycle):
                print(vehicle.do_wheelie())
            elif isinstance(vehicle, Bus):
                print(vehicle.announce_stop())
            elif isinstance(vehicle, PassengerCar):
                print(vehicle.info())
        elif action == "8":
            print(f"Возраст машины: {vehicle.age} лет")
        elif action == "9":
            print(f"Уровень топлива: {vehicle.fuel_level:.1f} л")
        elif action == "10":
            break         elif action == "0":
            print("Выход из программы.")
            exit()
        else:
            print("Неверный выбор")
```

Результат выполнения программы:

```
2. Проверить пробег
3. Показать информацию
4. Конвертировать пробег в мили
5. Рассчитать налог
6. Показать тип двигателя
7. Дополнительное действие
8. Показать возраст машины
9. Показать уровень топлива
10. Создать новое транспортное средство
0. Выйти из программы
Ваш выбор: 9
Уровень топлива: 50.0 л

МЕНЮ ДЕЙСТВИЙ: BMW
1. Проехать расстояние
2. Проверить пробег
3. Показать информацию
4. Конвертировать пробег в мили
5. Рассчитать налог
6. Показать тип двигателя
7. Дополнительное действие
8. Показать возраст машины
9. Показать уровень топлива
10. Создать новое транспортное средство
0. Выйти из программы
Ваш выбор: 8
Возраст машины: 1 лет
```

Рисунок 1 - Добавленные декораторы

```
Введите данные о транспорте:
Марка: BMW
Цвет: белый
Пробег (в км): 20000
Год выпуска: 2024
Уровень топлива (л): 50
Владелец: Я
Тип двигателя: Бензиновый

Транспортное средство создано: Марка: BMW, Цвет: белый, Пробег: 20000 км, Год выпуска: 2024, Топливо: 50.0 л, Владелец: Я, Тип двигателя: Бензиновый
```

Рисунок 2 - Добавленные свойства

5.Контрольные вопросы:

1. Какое назначение у свойств классов?

Свойства классов позволяют управлять доступом к данным объекта, обеспечивая инкапсуляцию и возможность добавлять логику при чтении или изменении значений.

2. Какое назначение у декораторов методов?

Декораторы методов используются для изменения или расширения поведения функций/методов без изменения их исходного кода, например для логирования, проверки прав доступа или кеширования.